IY4113 Milestone 4

Assessment Details	Please Complete All Details
Group	A
Module Title	Applied Software Engineering using Object-Orientated Programming
Assessment Type	coursework
Module Tutor Name	Jonathan Shore
Student ID Number	P4661445
Date of Submission	22/05/2026
Word Count	3820
GItHub Link	https://github.com/eugnx/T0466145_Eugene_4113

- [x] *I confirm that this assignment is my own work. Where I have referred to academic sources, I have provided in-text citations and included the sources in the final reference list.*
- [x] *Where I have used AI, I have cited and referenced appropriately.

Program Code

FareCalculator

```
import java.math.BigDecimal;
import java.math.RoundingMode;

public class FareCalculator {
//Calculates zones crossed using formula |to - from| + 1.
```

```
public static int calculateZonesCrossed(int fromZone, int toZone) {        int zones;
    zones = Math.abs(toZone - fromZone) + 1;
    return zones;
}

 // Gets base fare from CityRideDataset.

public static BigDecimal getBaseFare(int fromZone, int toZone, CityRideDataset.TimeBand t
    BigDecimal fare;
    fare = CityRideDataset.getBaseFare(fromZone, toZone, timeBand);

    if (fare == null) {
        fare = BigDecimal.ZERO;
    }

    return fare;
}

 // Applies discount to base fare.

public static BigDecimal applyDiscount(BigDecimal baseFare, BigDecimal discountRate) {
    BigDecimal discountAmount;
    BigDecimal discountedFare;

    discountAmount = baseFare.multiply(discountRate);
    discountedFare = baseFare.subtract(discountAmount);
    discountedFare = discountedFare.setScale(2, RoundingMode.HALF_UP);

    return discountedFare;
}

 // Calculates charged fare with daily cap consideration.

public static BigDecimal applyDailyCap(BigDecimal discountedFare, BigDecimal currentTotal
    BigDecimal newTotal;
    BigDecimal chargedFare;

    newTotal = currentTotal.add(discountedFare);

    if (newTotal.compareTo(dailyCap) > 0) {
        chargedFare = dailyCap.subtract(currentTotal);
        if (chargedFare.compareTo(BigDecimal.ZERO) < 0) {
            chargedFare = BigDecimal.ZERO;
        }
    } else {
        chargedFare = discountedFare;
    }

    chargedFare = chargedFare.setScale(2, RoundingMode.HALF_UP);
```

```
    return chargedFare;
}

 // Calculates the complete fare for a journey.

public static BigDecimal calculateFullFare(Journey journey, BigDecimal currentTotal) {
    int zonesCrossed;
    BigDecimal baseFare;
    BigDecimal discountedFare;
    BigDecimal dailyCap;
    BigDecimal chargedFare;

    // Calculate zones crossed
    zonesCrossed = journey.calculateZonesCrossed();
    journey.setZonesCrossed(zonesCrossed);

    // Get base fare from dataset
    baseFare = getBaseFare(journey.getFromZone(), journey.getToZone(), journey.getTimeBan
    journey.setBaseFare(baseFare);

    // Apply passenger discount
    discountedFare = applyDiscount(baseFare, journey.getDiscountRate());

    // Apply daily cap
    dailyCap = journey.getDailyCap();
    chargedFare = applyDailyCap(discountedFare, currentTotal, dailyCap);

    // Store discount rate and charged fare
    journey.setDiscountApplied(journey.getDiscountRate());
    journey.setChargedFare(chargedFare);

    return chargedFare;
```

}

IMPORT VALIDATOR

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;

public class InputValidator {
```

```
private static final DateTimeFormatter DATE_FORMATTER = DateTimeFormatter.ofPattern("d/M/
public static boolean isValidZone(int zone) {
    return zone >= CityRideDataset.MIN_ZONE && zone <= CityRideDataset.MAX_ZONE;
}

public static boolean isValidDate(String dateStr) {
    // try-catch is used to handle runtime errors so the program does not crash
    try {
        LocalDate.parse(dateStr, DATE_FORMATTER);
        return true;
    } catch (DateTimeParseException e) {
        return false;
    }
}

public static LocalDate parseDate(String dateStr) {
    try {
        return LocalDate.parse(dateStr, DATE_FORMATTER);
    } catch (DateTimeParseException e) {
        return null;
    }
}

public static boolean isValidInteger(String input) {
    if (input == null || input.trim().isEmpty()) {
        return false;
    }
    try {
        Integer.parseInt(input.trim()); // trim() removes any leading and trailing whites
        return true;
    } catch (NumberFormatException e) {
        return false;
    }
}

public static boolean  isNotBlank(String Input){
    boolean isValid;
    if(Input == null)
         isValid = false;
    return !Input.trim().isEmpty();
}
// separates integers from strings

public static Integer parseInteger(String input) {
    try {
        return Integer.parseInt(input.trim());
    } catch (NumberFormatException e) {
        return null;
    }
}
```

```
  public static boolean isValidTimeBand(String input) {
      boolean isValid = false;

      if (input != null) {
          String upperInput = input.trim().toUpperCase();
          isValid = upperInput.equals("PEAK") || upperInput.equals("OFF_PEAK");
      }

      return isValid;
```

}
IINPUTVALIDATOR

public static CityRideDataset.TimeBand parseTimeBand(String input) {
CityRideDataset.TimeBand band = null;

```
    if (input != null) {            String upperInput = input.trim().toUpperCase();
        if (upperInput.equals("PEAK")) {
            band = CityRideDataset.TimeBand.PEAK;
        } else if (upperInput.equals("OFF_PEAK")) {
            band = CityRideDataset.TimeBand.OFF_PEAK;
        }
    }

    return band;
}

public static boolean isValidPassengerType(String input) {
    boolean isValid = false;

    if (input != null) {
        String upperInput = input.trim().toUpperCase();
        isValid = upperInput.equals("ADULT") ||
                upperInput.equals("STUDENT") ||
                upperInput.equals("CHILD") ||
                upperInput.equals("SENIOR_CITIZEN");
    }

    return isValid;
}

public static CityRideDataset.PassengerType parsePassengerType(String input) {
    CityRideDataset.PassengerType type = null;

    if (input != null) {
        String upperInput = input.trim().toUpperCase();
        switch (upperInput) {
            case "ADULT":
                type = CityRideDataset.PassengerType.ADULT;
                break;
            case "STUDENT":
                type = CityRideDataset.PassengerType.STUDENT;
                break;
            case "CHILD":
                type = CityRideDataset.PassengerType.CHILD;
                break;
            case "SENIOR_CITIZEN":
                type = CityRideDataset.PassengerType.SENIOR_CITIZEN;
                break;
            default:
                type = null;
        }
    }

    return type;
```

}

JOURNEY

```
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class Journey {
```

```
// Private fields    private int journeyId;
private LocalDate date;
private CityRideDataset.TimeBand timeBand;
private int fromZone;
private int toZone;
private CityRideDataset.PassengerType passengerType;
private int zonesCrossed;
private BigDecimal baseFare;
private BigDecimal discountApplied;
private BigDecimal chargedFare;


 // Constructor for creating a new journey.

public Journey(int journeyId, LocalDate date, CityRideDataset.TimeBand timeBand,
               int fromZone, int toZone, CityRideDataset.PassengerType passengerType) {
    this.journeyId = journeyId;
    this.date = date;
    this.timeBand = timeBand;
    this.fromZone = fromZone;
    this.toZone = toZone;
    this.passengerType = passengerType;
    this.zonesCrossed = 0;
    this.baseFare = BigDecimal.ZERO;
    this.discountApplied = BigDecimal.ZERO;
    this.chargedFare = BigDecimal.ZERO;
}


 // Calculates zones crossed using formula: |toZone - fromZone| + 1

public int calculateZonesCrossed() {
    return Math.abs(toZone - fromZone) + 1;
}


 // Gets discount rate from CityRideDataset.

public BigDecimal getDiscountRate() {
    return CityRideDataset.DISCOUNT_RATE.get(passengerType);
}


 // Gets daily cap from CityRideDataset.

public BigDecimal getDailyCap() {
    return CityRideDataset.DAILY_CAP.get(passengerType);
}

// Getters and Setters
```

```
  public int getJourneyId() { return journeyId; }
  public void setJourneyId(int journeyId) { this.journeyId = journeyId; }

  public LocalDate getDate() { return date; }
  public void setDate(LocalDate date) { this.date = date; }

  public CityRideDataset.TimeBand getTimeBand() { return timeBand; }
  public void setTimeBand(CityRideDataset.TimeBand timeBand) { this.timeBand = timeBand; }

  public int getFromZone() { return fromZone; }
  public void setFromZone(int fromZone) { this.fromZone = fromZone; }

  public int getToZone() { return toZone; }
  public void setToZone(int toZone) { this.toZone = toZone; }

  public CityRideDataset.PassengerType getPassengerType() { return passengerType; }
  public void setPassengerType(CityRideDataset.PassengerType passengerType) { this.passenge

  public int getZonesCrossed() { return zonesCrossed; }
  public void setZonesCrossed(int zonesCrossed) { this.zonesCrossed = zonesCrossed; }

  public BigDecimal getBaseFare() { return baseFare; }
  public void setBaseFare(BigDecimal baseFare) { this.baseFare = baseFare; }

  public BigDecimal getDiscountApplied() { return discountApplied; }
  public void setDiscountApplied(BigDecimal discountApplied) { this.discountApplied = disco

  public BigDecimal getChargedFare() { return chargedFare; }
  public void setChargedFare(BigDecimal chargedFare) { this.chargedFare = chargedFare; }


   // Returns formatted string representation of the journey.

  public String toString() {
      DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
      String fomattdate = date.format(formatter);

      double discountPercentage = discountApplied != null ? discountApplied.doubleValue() *
      return String.format("ID: %d | Date: %s | From: %d | To: %d | Time: %s | " +
                      "Passenger: %s | Zones: %d | Base: £%.2f | Charged: £%.2f",
              journeyId, date.format(formatter), fromZone, toZone, timeBand,
              passengerType, zonesCrossed, baseFare, chargedFare);


}
```

JOURNEYMANAGER

```
import java.math.BigDecimal;
import java.math.RoundingMode;
```

```
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class JourneyManager {
```

```
// Private fields    private List<Journey> journeys;
private int nextJourneyId;

// Running totals per passenger type
private Map<CityRideDataset.PassengerType, BigDecimal> runningTotals;
private Map<CityRideDataset.PassengerType, Boolean> capReached;

// Tracking for detailed reporting
private Map<CityRideDataset.PassengerType, BigDecimal> preDiscountTotals;
private Map<CityRideDataset.PassengerType, BigDecimal> discountedTotals;
private Map<CityRideDataset.PassengerType, Integer> journeyCountByType;


public JourneyManager() {
    this.journeys = new ArrayList<>();
    this.nextJourneyId = 1;
    this.runningTotals = new HashMap<>();
    this.capReached = new HashMap<>();
    this.preDiscountTotals = new HashMap<>();
    this.discountedTotals = new HashMap<>();
    this.journeyCountByType = new HashMap<>();

    // Initialize for each passenger type
    for (CityRideDataset.PassengerType type : CityRideDataset.PassengerType.values()) {
        runningTotals.put(type, BigDecimal.ZERO);
        capReached.put(type, false);
        preDiscountTotals.put(type, BigDecimal.ZERO);
        discountedTotals.put(type, BigDecimal.ZERO);
        journeyCountByType.put(type, 0);
    }
}


public Journey addJourney(Journey journey) {
    CityRideDataset.PassengerType passengerType;
    BigDecimal currentTotal;
    BigDecimal chargedFare;
    boolean capAlreadyReached;

    passengerType = journey.getPassengerType();
    currentTotal = runningTotals.get(passengerType);
    capAlreadyReached = capReached.get(passengerType);

    // Get base fare for pre-discount tracking
    BigDecimal baseFare = FareCalculator.getBaseFare(
            journey.getFromZone(), journey.getToZone(), journey.getTimeBand()
    );

    // Update pre-discount total
    BigDecimal newPreDiscountTotal = preDiscountTotals.get(passengerType).add(baseFare);
```

```
    preDiscountTotals.put(passengerType, newPreDiscountTotal);

    // Update journey count
    journeyCountByType.put(passengerType, journeyCountByType.get(passengerType) + 1);

    // Check if cap already reached
    if (capAlreadyReached) {
        chargedFare = BigDecimal.ZERO;
        journey.setChargedFare(chargedFare);
        journey.setZonesCrossed(journey.calculateZonesCrossed());
        journey.setBaseFare(baseFare);
        journey.setDiscountApplied(journey.getDiscountRate());
    } else {
        // Calculate fare with cap consideration
        chargedFare = FareCalculator.calculateFullFare(journey, currentTotal);

        // Update discounted total for reporting
        BigDecimal discountedFare = FareCalculator.applyDiscount(baseFare, journey.getDis
        BigDecimal newDiscountedTotal = discountedTotals.get(passengerType).add(discounte
        discountedTotals.put(passengerType, newDiscountedTotal);
    }

    // Set journey ID
    journey.setJourneyId(nextJourneyId);
    nextJourneyId++;

    // Update running total
    BigDecimal newTotal = runningTotals.get(passengerType).add(chargedFare);
    runningTotals.put(passengerType, newTotal);

    // Check if cap is now reached
    if (newTotal.compareTo(journey.getDailyCap()) >= 0) {
        capReached.put(passengerType, true);
    }

    // Add to list
    journeys.add(journey);

    return journey;
}

public List<Journey> getAllJourneys() {
    return new ArrayList<>(journeys);
}


public Journey getJourneyById(int id) {
    Journey foundJourney = null;

    for (Journey journey : journeys) {
        if (journey.getJourneyId() == id) {
```

```
            foundJourney = journey;
            break;
        }
    }
    return foundJourney;
}


public boolean removeJourneyById(int id) {
    Journey journeyToRemove;
    boolean removed;

    journeyToRemove = getJourneyById(id);

    if (journeyToRemove == null) {
        removed = false;
    } else {
        journeys.remove(journeyToRemove);
        recalculateAllTotals();
        removed = true;
    }

    return removed;
}

private void recalculateAllTotals() {
    // Reset all totals and cap flags
    for (CityRideDataset.PassengerType type : CityRideDataset.PassengerType.values()) {
        runningTotals.put(type, BigDecimal.ZERO);
        capReached.put(type, false);
        preDiscountTotals.put(type, BigDecimal.ZERO);
        discountedTotals.put(type, BigDecimal.ZERO);
        journeyCountByType.put(type, 0);
    }

    // Re-add each journey in order (recalculate fares)
    for (Journey journey : journeys) {
        CityRideDataset.PassengerType passengerType = journey.getPassengerType();
        BigDecimal currentTotal = runningTotals.get(passengerType);
        boolean capAlreadyReached = capReached.get(passengerType);

        // Update pre-discount total
        BigDecimal newPreDiscountTotal = preDiscountTotals.get(passengerType).add(journey
        preDiscountTotals.put(passengerType, newPreDiscountTotal);

        // Update journey count
        journeyCountByType.put(passengerType, journeyCountByType.get(passengerType) + 1);

        // Update discounted total
        BigDecimal discountedFare = FareCalculator.applyDiscount(journey.getBaseFare(), j
        BigDecimal newDiscountedTotal = discountedTotals.get(passengerType).add(discounte
```

```
        discountedTotals.put(passengerType, newDiscountedTotal);

        BigDecimal chargedFare;
        if (capAlreadyReached) {
            chargedFare = BigDecimal.ZERO;
        } else {
            chargedFare = FareCalculator.applyDailyCap(discountedFare, currentTotal, jour
        }

        journey.setChargedFare(chargedFare);

        BigDecimal newTotal = runningTotals.get(passengerType).add(chargedFare);
        runningTotals.put(passengerType, newTotal);

        if (newTotal.compareTo(journey.getDailyCap()) >= 0) {
            capReached.put(passengerType, true);
        }
    }
}

public void resetDay() {
    journeys.clear();
    nextJourneyId = 1;

    for (CityRideDataset.PassengerType type : CityRideDataset.PassengerType.values()) {
        runningTotals.put(type, BigDecimal.ZERO);
        capReached.put(type, false);
        preDiscountTotals.put(type, BigDecimal.ZERO);
        discountedTotals.put(type, BigDecimal.ZERO);
        journeyCountByType.put(type, 0);
    }
}


public int getJourneyCount() {
    return journeys.size();
}

public BigDecimal getPassengerTotal(CityRideDataset.PassengerType passengerType) {
    return runningTotals.get(passengerType);
}

public BigDecimal getPreDiscountTotalByType(CityRideDataset.PassengerType passengerType)
    return preDiscountTotals.get(passengerType);
}

public BigDecimal getDiscountedTotalByType(CityRideDataset.PassengerType passengerType) {
    return discountedTotals.get(passengerType);
}

public int getJourneyCountByType(CityRideDataset.PassengerType passengerType) {
```

```
    return journeyCountByType.get(passengerType);
}

public boolean isCapReached(CityRideDataset.PassengerType passengerType) {
    return capReached.get(passengerType);
}

public BigDecimal getTotalCost() {
    BigDecimal total = BigDecimal.ZERO;

    for (BigDecimal amount : runningTotals.values()) {
        total = total.add(amount);
    }

    return total;
}

public BigDecimal getAverageCostPerJourney() {
    int count = journeys.size();
    if (count == 0) {
        return BigDecimal.ZERO;
    }
    BigDecimal total = getTotalCost();
    BigDecimal average = total.divide(new BigDecimal(count), 2, RoundingMode.HALF_UP);

    return average;
}

public Journey getMostExpensiveJourney() {
    Journey mostExpensive = null;

    for (Journey journey : journeys) {
        if (mostExpensive == null ||
                journey.getChargedFare().compareTo(mostExpensive.getChargedFare()) > 0) {
            mostExpensive = journey;
        }
    }

    return mostExpensive;
}


public Map<String, Integer> getTimeBandCounts() {
    Map<String, Integer> counts = new HashMap<>();
    counts.put("PEAK", 0);
    counts.put("OFF_PEAK", 0);

    for (Journey journey : journeys) {
        if (journey.getTimeBand() == CityRideDataset.TimeBand.PEAK) {
            counts.put("PEAK", counts.get("PEAK") + 1);
        } else {
```

```
            counts.put("OFF_PEAK", counts.get("OFF_PEAK") + 1);
        }
    }

    return counts;


}
```

}
//(deepseek 2026)
public Map<String, Integer> getZoneUsage() {
Map<String, Integer> zoneUsage = new HashMap<>();

```
    for (Journey journey : journeys) {            String fromKey = "From Zone " + journey
        String toKey = "To Zone " + journey.getToZone();

        zoneUsage.put(fromKey, zoneUsage.getOrDefault(fromKey, 0) + 1);
        zoneUsage.put(toKey, zoneUsage.getOrDefault(toKey, 0) + 1);
    }

    return zoneUsage;
}

// FILTERING METHODS

public List<Journey> getJourneysByPassengerType(CityRideDataset.PassengerType type) {
    List<Journey> filtered = new ArrayList<>();

    for (Journey journey : journeys) {
        if (journey.getPassengerType() == type) {
            filtered.add(journey);
        }
    }

    return filtered;
}

public List<Journey> getJourneysByTimeBand(CityRideDataset.TimeBand timeBand) {
    List<Journey> filtered = new ArrayList<>();

    for (Journey journey : journeys) {
        if (journey.getTimeBand() == timeBand) {
            filtered.add(journey);
        }
    }

    return filtered;
}


public List<Journey> getJourneysByFromZone(int zone) {
    List<Journey> filtered = new ArrayList<>();

    for (Journey journey : journeys) {
        if (journey.getFromZone() == zone) {
            filtered.add(journey);
        }
    }

    return filtered;
}
```

```
  public List<Journey> getJourneysByToZone(int zone) {
      List<Journey> filtered = new ArrayList<>();

      for (Journey journey : journeys) {
          if (journey.getToZone() == zone) {
              filtered.add(journey);
          }
      }

      return filtered;
  }


  public List<Journey> getJourneysByZone(int zone) {
      List<Journey> filtered = new ArrayList<>();

      for (Journey journey : journeys) {
          if (journey.getFromZone() == zone || journey.getToZone() == zone) {
              filtered.add(journey);
          }
      }

      return filtered;
  }

  public List<Journey> getJourneysByDate(LocalDate date) {
      List<Journey> filtered = new ArrayList<>();

      for (Journey journey : journeys) {
          if (journey.getDate().equals(date)) {
              filtered.add(journey);
          }
      }

      return filtered;
```

}

**MAIN **

public class Main {

```
  public static void main(String[] args) {        MenuHandler menuHandler = new MenuHandler
      menuHandler.start();
  }
```

}

MENUHANDLER

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;
import java.util.Scanner;
import java.util.Map;

public class MenuHandler {

```
private static final int MENU_ADD_JOURNEY = 1;    private static final int MENU_LIST_JOUR
private static final int MENU_FILTER_JOURNEYS = 3;
private static final int MENU_DAILY_SUMMARY = 4;
private static final int MENU_TOTALS_BY_TYPE = 5;
private static final int MENU_REMOVE_JOURNEY= 6;
private static final int MENU_RESET_DAY = 7;
private static final int MENU_EXIT = 8;

// sub-menu constants
private static final int FILTER_BY_PASSENGER_TYPE = 1;
private static final int FILTER_BY_TIME_BAND = 2;
private static final int FILTER_BY_ZONE = 3;
private static final int FILTER_BY_DATE = 4;
```

private static final int FILTER_BACK_TO_MAIN = 5;
// private fileds
private Scanner scanner;
private JourneyManager journeyManager;
private boolean running;

```
public MenuHandler() {        this.scanner = new Scanner(System.in);
    this.journeyManager = new JourneyManager();
    this.running = true;
}

public void start() {
    displayWelcomeMessage();

    while (running) {
        displayMainMenu();
        int choice = getIntInput("Enter your choice: ");
        processMenuChoice(choice);
    }

    System.out.println("\nThank you for using CityRide Lite!");
    scanner.close(); //  used to close the Scanner object when you are finished using it
}

private void displayWelcomeMessage() {
    System.out.println("\n========================================");
    System.out.println("     WELCOME TO CITYRIDE LITE");
    System.out.println("========================================");
}

private void displayMainMenu() {
    System.out.println("\n========== MAIN MENU ==========");
    System.out.println("1. Add Journey");
    System.out.println("2. List All Journeys");
    System.out.println("3. Filter Journeys");
    System.out.println("4. View Daily Summary");
    System.out.println("5. View Totals by Passenger Type");
    System.out.println("6. Remove Journey");
    System.out.println("7. Reset Day");
    System.out.println("8. Exit");
    System.out.println("================================");
}

private void processMenuChoice(int choice) {
    switch (choice) {
        case MENU_ADD_JOURNEY:
            addJourney();
            break;

        case MENU_LIST_JOURNEYS:
            listAllJourneys();
            break;

        case MENU_FILTER_JOURNEYS:
            filterJourneys();
            break;
```

```
        case MENU_DAILY_SUMMARY:
            displayDailySummary();
            break;

        case MENU_TOTALS_BY_TYPE:
            displayTotalsByType();
            break;

        case MENU_REMOVE_JOURNEY:
            removeJourney();
            break;

        case MENU_RESET_DAY:
            resetDay();
            break;

        case MENU_EXIT:
            running = false;
            break;

        default:
            System.out.println("Invalid choice! Please enter 1-8.");
    }
}


private void addJourney() {
    System.out.println("\n--- Add New Journey ---");

    LocalDate date = getDateInput("Enter date (DD/MM/YYYY) or press Enter for today: ");
    CityRideDataset.TimeBand timeBand = getTimeBandInput();
    int fromZone = getZoneInput("Enter starting zone (1-5): ");
    int toZone = getZoneInput("Enter destination zone (1-5): ");
    CityRideDataset.PassengerType passengerType = getPassengerTypeInput();

    Journey journey = new Journey(0, date, timeBand, fromZone, toZone, passengerType);
    Journey completedJourney = journeyManager.addJourney(journey);

    System.out.println("\n✓ Journey added successfully!");
    System.out.println("  Journey ID: " + completedJourney.getJourneyId());
    System.out.printf("  Zones crossed: %d%n", completedJourney.getZonesCrossed());
    System.out.printf("  Base fare: £%.2f%n", completedJourney.getBaseFare());
    System.out.printf("  Charged fare: £%.2f%n", completedJourney.getChargedFare());

    if (journeyManager.isCapReached(passengerType)) {
        System.out.println("  ⚠ Daily cap reached for " + passengerType + "!");
    }
}

private void listAllJourneys() {
```

```
    List<Journey> journeys = journeyManager.getAllJourneys();

    System.out.println("\n--- All Journeys ---");

    if (journeys.isEmpty()) {
        System.out.println("No journeys recorded yet.");
    } else {
        System.out.println("\nTotal journeys: " + journeys.size());
        System.out.println("-".repeat(80));

        for (Journey journey : journeys) {
            System.out.println(journey);
        }

        System.out.println("-".repeat(80));
        System.out.printf("Total cost for the day: £%.2f%n", journeyManager.getTotalCost(
    }
}

private void filterJourneys() {
    boolean filterMenuActive = true;

    while (filterMenuActive == true) {
        displayFilterMenu();
        int choice = getIntInput("Enter your choice: ");

        if (choice == FILTER_BY_PASSENGER_TYPE) {
            filterByPassengerType();
        } else if (choice == FILTER_BY_TIME_BAND) {
            filterByTimeBand();
        } else if (choice == FILTER_BY_ZONE) {
            filterByZone();
        } else if (choice == FILTER_BY_DATE) {
            filterByDate();
        } else if (choice == FILTER_BACK_TO_MAIN) {
            filterMenuActive = false;
        } else {
            System.out.println("Invalid choice! Please enter 1-5.");
        }
    }
}

private void displayFilterMenu() {
    System.out.println("\n========== FILTER MENU ==========");
    System.out.println("1. Filter by Passenger Type");
    System.out.println("2. Filter by Time Band");
    System.out.println("3. Filter by Zone (from or to)");
    System.out.println("4. Filter by Date");
    System.out.println("5. Back to Main Menu");
    System.out.println("================================");
}
```

```
private void filterByPassengerType() {
    System.out.println("\n--- Filter by Passenger Type ---");
    System.out.println("Available types: ADULT, STUDENT, CHILD, SENIOR_CITIZEN");

    CityRideDataset.PassengerType type = getPassengerTypeInput();
    List<Journey> filtered = journeyManager.getJourneysByPassengerType(type);

    displayFilterResults(filtered, "Passenger Type: " + type);
}

private void filterByTimeBand() {
    System.out.println("\n--- Filter by Time Band ---");

    CityRideDataset.TimeBand band = getTimeBandInput();
    List<Journey> filtered = journeyManager.getJourneysByTimeBand(band);

    displayFilterResults(filtered, "Time Band: " + band);
}

private void filterByZone() {
    System.out.println("\n--- Filter by Zone ---");
    int zone = getZoneInput("Enter zone number (1-5): ");

    System.out.println("Filter by:");
    System.out.println("1. Starting zone (from)");
    System.out.println("2. Destination zone (to)");
    System.out.println("3. Either zone");

    int choice = getIntInput("Enter choice: ");

    List<Journey> filtered = null;

    if (choice == 1) {
        filtered = journeyManager.getJourneysByFromZone(zone);
    } else if (choice == 2) {
        filtered = journeyManager.getJourneysByToZone(zone);
    } else if (choice == 3) {
        filtered = journeyManager.getJourneysByZone(zone);
    } else {
        System.out.println("Invalid choice!");
        return;
    }

    displayFilterResults(filtered, "Zone: " + zone);
}

private void filterByDate() {
    System.out.println("\n--- Filter by Date ---");
    LocalDate date = getDateInput("Enter date (DD/MM/YYYY): ");
    List<Journey> filtered = journeyManager.getJourneysByDate(date);
```

```
    displayFilterResults(filtered, "Date: " + date.toString());
}

private void displayFilterResults(List<Journey> journeys, String filterCriteria) {
    System.out.println("\n--- Filter Results (" + filterCriteria + ") ---");

    if (journeys == null || journeys.isEmpty()) {
        System.out.println("No journeys match this filter.");
    } else {
        System.out.println("Found " + journeys.size() + " journey(s):");
        System.out.println("-".repeat(90));

        for (Journey journey : journeys) {
            System.out.println(journey);
        }

        // Calculate total for filtered journeys
        BigDecimal total = BigDecimal.ZERO;
        for (Journey journey : journeys) {
            total = total.add(journey.getChargedFare());
        }
        System.out.println("-".repeat(90));
        System.out.printf("Total cost for filtered journeys: £%.2f%n", total);
    }
}

private void displayDailySummary() {
    List<Journey> journeys = journeyManager.getAllJourneys();
    int totalJourneys = journeys.size();

    System.out.println("\n========== DAILY SUMMARY ==========");

    if (totalJourneys == 0) {
        System.out.println("No journeys recorded today.");
    } else {
        BigDecimal totalCost = journeyManager.getTotalCost();
        BigDecimal averageCost = journeyManager.getAverageCostPerJourney();
        Journey mostExpensive = journeyManager.getMostExpensiveJourney();

        System.out.println("Total number of journeys: " + totalJourneys);
        System.out.printf("Total cost charged: £%.2f%n", totalCost);
        System.out.printf("Average cost per journey: £%.2f%n", averageCost);

        if (mostExpensive != null) {
            System.out.printf("Most expensive journey: Journey #%d (£%.2f)%n",
                    mostExpensive.getJourneyId(), mostExpensive.getChargedFare());
        }

        // Category counts
        displayCategoryCounts();
```

```
    }

}

private void displayCategoryCounts() {
    Map<String, Integer> timeBandCounts = journeyManager.getTimeBandCounts();
    Map<String, Integer> zoneUsage = journeyManager.getZoneUsage();

    System.out.println("\n--- Category Counts ---");
    System.out.println("Peak journeys: " + timeBandCounts.get("PEAK"));
    System.out.println("Off-peak journeys: " + timeBandCounts.get("OFF_PEAK"));

    System.out.println("\nZone usage:");
    if (zoneUsage.isEmpty()) {
        System.out.println("  No zone usage data available.");
    } else {
        for (Map.Entry<String, Integer> entry : zoneUsage.entrySet()) {
            System.out.println("  " + entry.getKey() + ": " + entry.getValue() + " time(s
        }
    }
}

private void displayTotalsByType() {
    System.out.println("\n========== TOTALS BY PASSENGER TYPE ==========");

    // Header
    System.out.printf("%-15s %10s %10s %10s %10s %12s%n",
            "Type", "Journeys", "Pre-Discount", "Discounted", "Charged", "Cap Reached");
    System.out.println("-".repeat(75));

    for (CityRideDataset.PassengerType type : CityRideDataset.PassengerType.values()) {
        int journeyCount = journeyManager.getJourneyCountByType(type);
        BigDecimal preDiscountTotal = journeyManager.getPreDiscountTotalByType(type);
        BigDecimal discountedTotal = journeyManager.getDiscountedTotalByType(type);
        BigDecimal chargedTotal = journeyManager.getPassengerTotal(type);
        boolean capReached = journeyManager.isCapReached(type);

        System.out.printf("%-15s %10d £%8.2f £%8.2f £%8.2f %12s%n",
                type, journeyCount, preDiscountTotal, discountedTotal, chargedTotal,
                (capReached ? "YES" : "NO"));
    }
    System.out.println("=".repeat(75));
}

private void removeJourney() {
    System.out.println("\n--- Remove Journey ---");

    if (journeyManager.getJourneyCount() == 0) {
        System.out.println("No journeys to remove.");
        return;
    }
```

```
    listAllJourneys();
    int id = getIntInput("Enter journey ID to remove: ");

    Journey journeyToRemove = journeyManager.getJourneyById(id);

    if (journeyToRemove == null) {
        System.out.println("Error: Journey with ID " + id + " not found!");
    } else {
        System.out.print("Are you sure you want to remove journey " + id + "? (Y/N): ");
        String confirm = scanner.nextLine().trim().toUpperCase();

        if (confirm.equals("Y")) {
            boolean removed = journeyManager.removeJourneyById(id);
            if (removed) {
                System.out.println("Journey " + id + " removed successfully!");
                System.out.println("All totals have been recalculated.");
            } else {
                System.out.println("Error: Could not remove journey.");
            }
        } else {
            System.out.println("Removal cancelled.");
        }
    }
}

private void resetDay() {
    System.out.println("\n--- Reset Day ---");
    System.out.print("WARNING: This will delete ALL journeys. Are you sure? (Y/N): ");
    String confirm = scanner.nextLine().trim().toUpperCase();

    if (confirm.equals("Y")) {
        journeyManager.resetDay();
        System.out.println("✓ Day reset successfully! All journeys cleared.");
    } else {
        System.out.println("Reset cancelled.");
    }
}



private LocalDate getDateInput(String prompt) {
    boolean valid = false;
    String input = "";

    while (valid == false) {
        System.out.print(prompt);
        input = scanner.nextLine().trim();

        if (input.isEmpty()) {
            return LocalDate.now();
```

```
        }

        if (InputValidator.isValidDate(input)) {
            valid = true;
        } else {
            System.out.println("Error: Invalid date format! Use DD/MM/YYYY");
        }
    }

    return InputValidator.parseDate(input);
}

private CityRideDataset.TimeBand getTimeBandInput() {
    boolean valid = false;
    CityRideDataset.TimeBand band = null;

    while (valid == false) {
        System.out.print("Enter time band (PEAK / OFF_PEAK): ");
        String input = scanner.nextLine().trim().toUpperCase();

        if (InputValidator.isValidTimeBand(input)) {
            band = InputValidator.parseTimeBand(input);
            valid = true;
        } else {
            System.out.println("Error: Invalid time band! Enter PEAK or OFF_PEAK");
        }
    }
    return band;
}


private int getZoneInput(String prompt) {
    boolean valid = false;
    int zone = -1;

    while (valid == false) {
        zone = getIntInput(prompt);

        if (InputValidator.isValidZone(zone)) {
            valid = true;
        } else {
            System.out.println("Error: Zone must be between 1 and 5");
        }
    }

    return zone;
}

private CityRideDataset.PassengerType getPassengerTypeInput() {
    boolean valid = false;
    CityRideDataset.PassengerType type = null;
```

```
        while (valid == false) {
            System.out.print("Enter passenger type (ADULT/STUDENT/CHILD/SENIOR_CITIZEN): ");
            String input = scanner.nextLine().trim().toUpperCase();

            if (InputValidator.isValidPassengerType(input)) {
                type = InputValidator.parsePassengerType(input);
                valid = true;
            } else {
                System.out.println("Error: Invalid passenger type!");
            }
        }

        return type;
    }


    private int getIntInput(String prompt) {
        boolean valid = false;
        int value = 0;

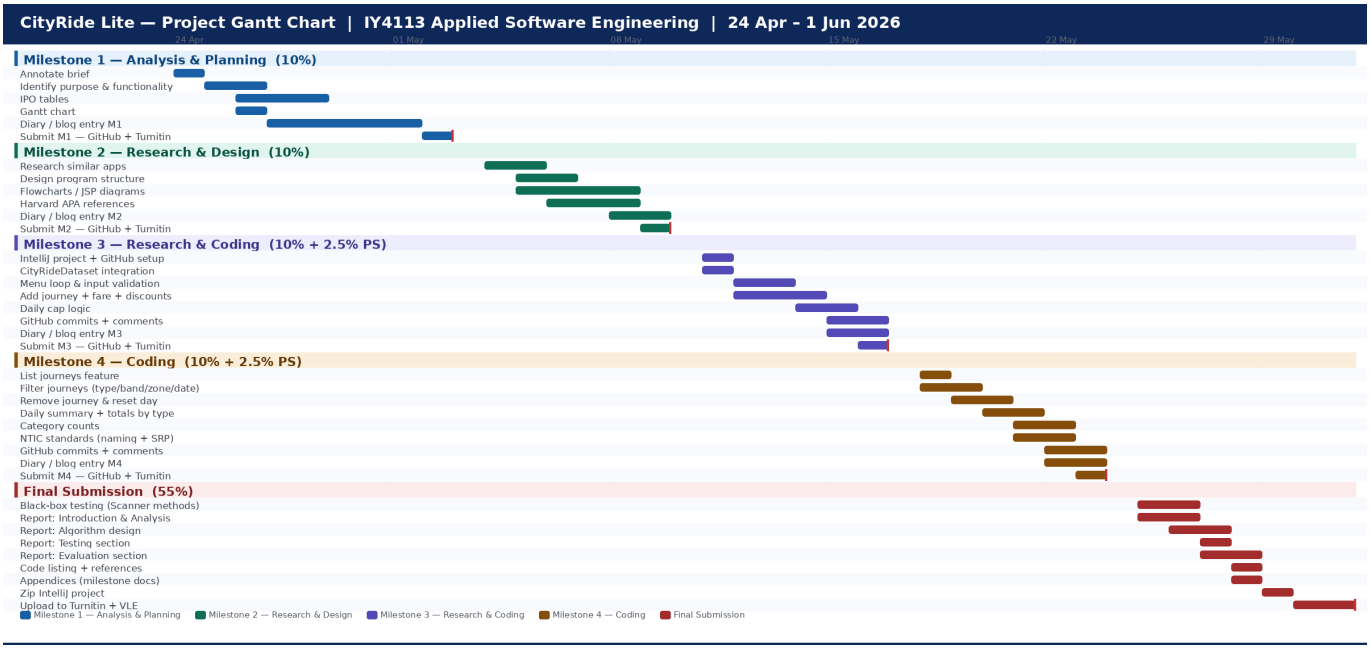
        while (valid == false) {
            System.out.print(prompt);
            String input = scanner.nextLine().trim();

            if (input.isEmpty()) {
                System.out.println("Error: Input cannot be blank!");
            } else if (InputValidator.isValidInteger(input)) {
                value = Integer.parseInt(input);
                valid = true;
            } else {
                System.out.println("Error: Please enter a valid number!");
            }
        }

        return value;
```

}
}

Updated Gantt Chart



Diary Entries

Entry 1 – Date [18/05/2026] Completing the reset of features

In this session, I added more features to the program. I finished making it possible to filter journeys, remove journeys, reset the system, and show journey summaries. I also made the menu better so users can move through the program more easily.
I changed the fare system so it now uses the given dataset for fares, discounts, and daily caps instead of fixed numbers. I had trouble with the daily cap because some fares were going above the limit after many journeys were added. I fixed this by improving how fares are calculated and by using BigDecimal to make money calculations more accurate.

Entry 2 – Date [20/05/2026] Testing the program

I tested all the main features of the program using different inputs. I checked valid and invalid zones, passenger types, peak and off-peak fares, daily caps, removing journeys, and filtering journeys. I also created testing tables to record the results.

Some invalid inputs caused the program to show the wrong messages or skip parts of the input process. To fix this, I improved the validation and added better checks. This stopped the program from crashing when wrong input was entered.

Entry 3 – Date [20/05/2026] Problems encounted and solving them

During Milestone 4, I faced several major problems that required careful thinking. The hardest problem was correctly implementing the recalculateAllTotals() method. When a journey is removed, the totals for all later journeys may change because the daily cap might no longer apply in the same way. In my first attempt, I simply subtracted the removed journey's fare from the totals. This was wrong because, after removal, the daily cap might no longer be reached. I realized I needed to rebuild all totals from scratch by going through the remaining journeys in order. Fixing this took time, but it was the right way to solve the problem.

Another challenge was making sure all money values were rounded consistently to two decimal places. I fixed this by using BigDecimal.setScale(2, RoundingMode.HALF_UP) in every fare calculation method. W3Schools explains that BigDecimal is good for precise money calculations, and setScale() helps control decimal places (W3Schools, "Java BigDecimal"). Using RoundingMode.HALF_UP means numbers are rounded up when the next digit is 5 or more, which is the standard way to round money (W3Schools, "Java Math").

I also had to be careful with null checks when getting values from maps. If I tried to use a BigDecimal that was null, the program would crash with a NullPointerException. W3Schools notes that a NullPointerException happens when you try to use an object that doesn't exist (W3Schools, "Java NullPointerException"). To prevent this, I added proper null checks throughout the code.

References

W3Schools. "Java BigDecimal." W3Schools.com. [Online]. Available: https://www.w3schools.com/java/java_ref_bigdecimal.asp (Accessed 20 May)

W3Schools. "Java Math." W3Schools.com. [Online]. Available: https://www.w3schools.com/java/java_math.asp (Accessed 20 May)

W3Schools. "Java NullPointerException." W3Schools.com. [Online]. Available: https://www.w3schools.com/java/java_ref_errors_nullpointerexception.asp(Accessed 20 May)